

LAPORAN PRAKTIKUM

Internet of Think II



Oleh :

Nama : Nabil Hasani

NPM : 25782077

Kelas : TRI (3C)

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG TAHUN

2026

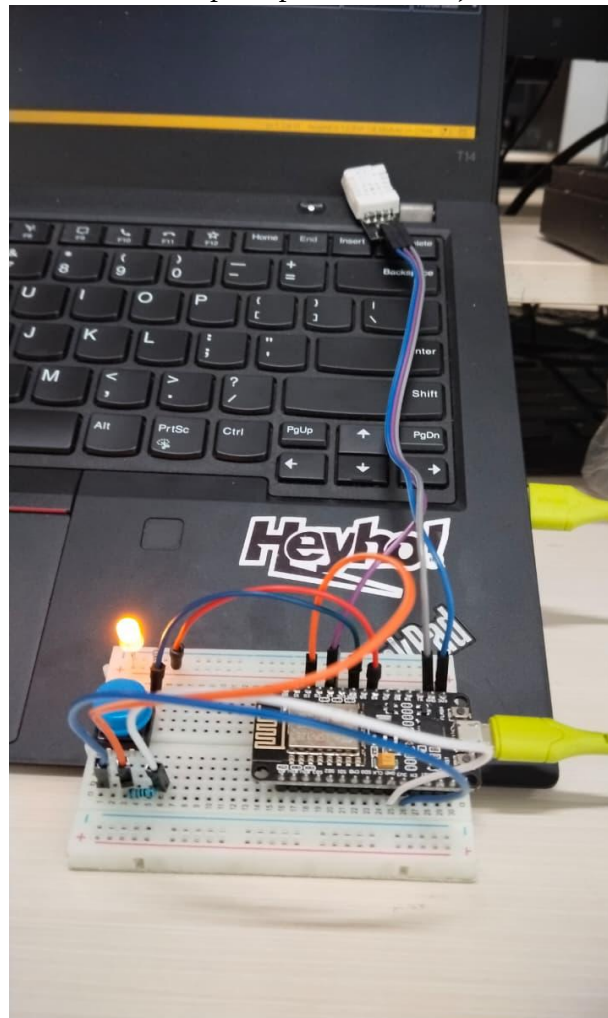
4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

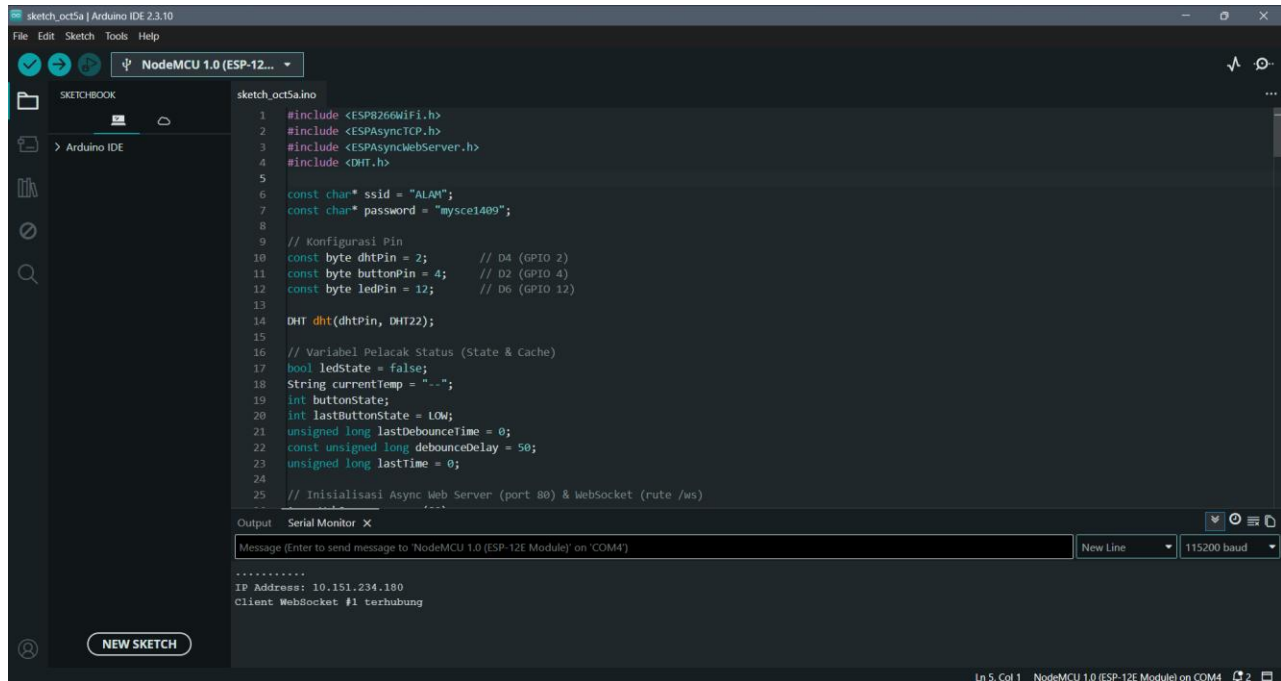
Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND NodeMCU** (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT DHT** ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).



Langkah Kerja Pemrograman: Kode program ini cukup kompleks karena menggabungkan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi `millis()`.

1. Buat berkas baru (*New Sketch*). Tuliskan kode program berikut.



```
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "ALAN";
7 const char* password = "mysce1409";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
24
25 // Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
```

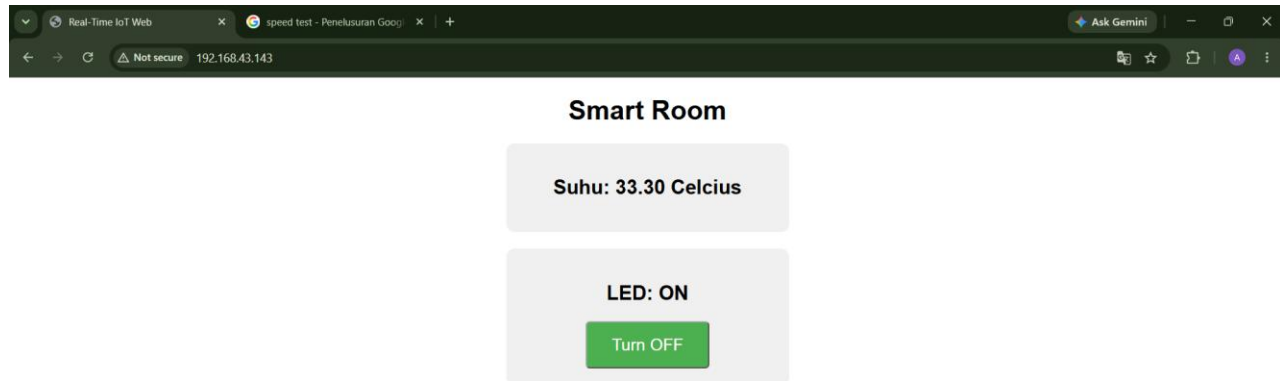
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

IP Address: 10.151.234.180
Client WebSocket #1 terhubung

2. Upload program. Pastikan IP Address didapatkan dari Serial Monitor.
3. Buka IP Address tersebut di browser laptop Anda, dan sekaligus buka IP yang sama di browser ponsel (smartphone) Anda secara berdampingan.
4. Uji 1 (Kendali Web Tersinkronisasi): Klik tombol pada browser laptop Anda. Perhatikan bagaimana browser di ponsel Anda ikut berubah (Turn ON) secara presisi, dan lampu LED fisik ikut menyala!

5. Uji 2 (Kendali Perangkat Keras): Sentuh push button mekanik di meja Anda. Amati bahwa antarmuka browser di laptop maupun ponsel Anda secara serentak berganti status (Turn OFF menjadi Turn ON) tanpa terjadi proses loading atau refresh HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman Asynchronous dan pemisahan (decoupling) pembacaan memori sensor DHT.



4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

- **Lebih Hemat Bandwidth:** Meta Refresh HTTP mengunduh ulang *seluruh* berkas halaman (HTML, CSS, header) tiap kali *refresh*. *WebSockets* hanya mengirimkan paket data kecil (seperti teks JSON) lewat satu jalur koneksi yang terus terbuka, tanpa perlu unduh ulang kerangka web.
- **Layar Tidak Berkedip:** Meta Refresh membongkar dan memuat ulang seluruh halaman dari nol, sehingga timbul kedipan. *WebSockets* memperbarui data secara langsung di belakang layar menggunakan JavaScript (*DOM manipulation*), jadi tampilan tetap mulus tanpa proses *loading*

Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

`delay(3000)` menghancurkan sifat *asynchronous* dan *real-time* dari *WebSockets*. Itulah mengapa pada arsitektur ini pemrosesan jeda waktu **wajib** menggunakan teknik *non-blocking* dengan menghitung selisih waktu fungsi `millis()`.

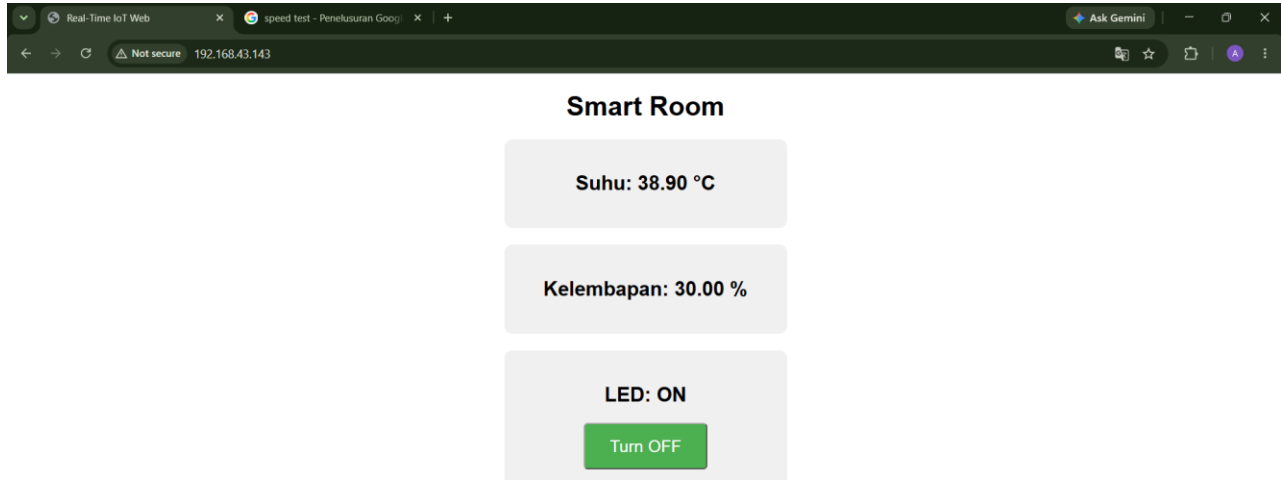
Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

1. **ledStatus** (pada tag `<span id="ledStatus">OFF</span>`)
 - **Target Pengubahan:** Diinjeksikan teks "ON" atau "OFF" melalui sintaks `status.innerHTML` untuk menampilkan status fisik LED secara *real-time*.
2. **toggleBtn** (pada tag `<button id="toggleBtn">...</button>`)*
 - **Target Pengubahan:** Diinjeksikan teks "Turn OFF" atau "Turn ON" melalui sintaks `btn.innerHTML`, serta diubah kelas CSS-nya (`btn-on` / `btn-off`) sesuai status LED.

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via *WebSockets*:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led": "...", "suhu": "...", "hum": "..."}.`

- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!



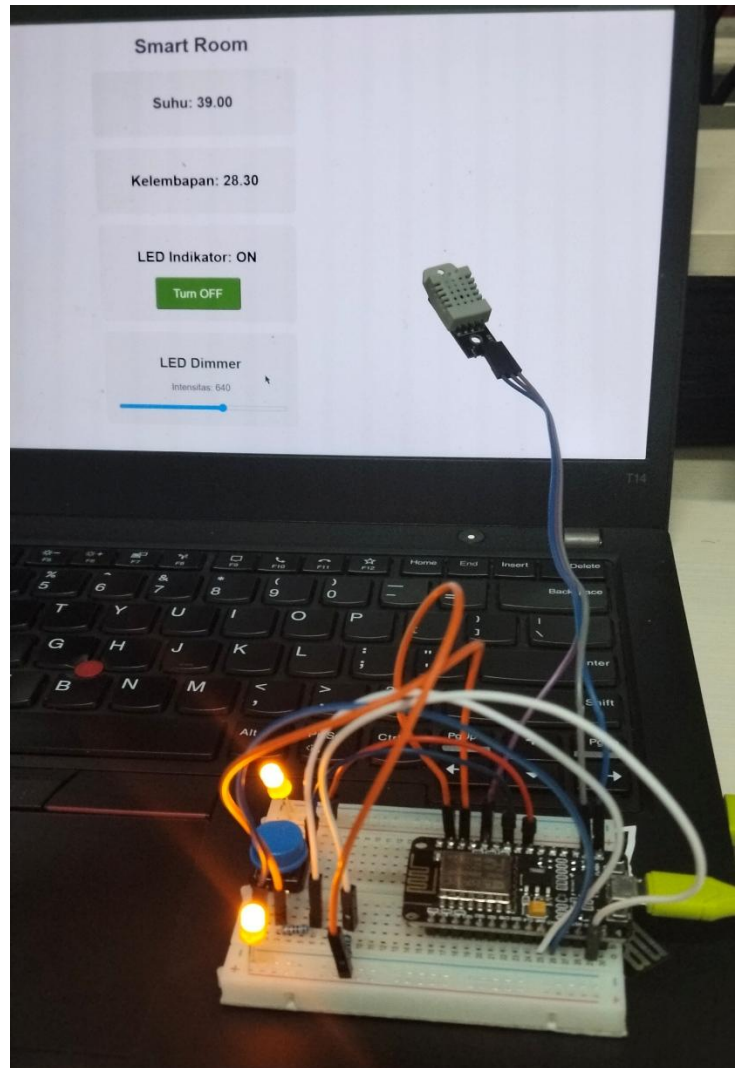
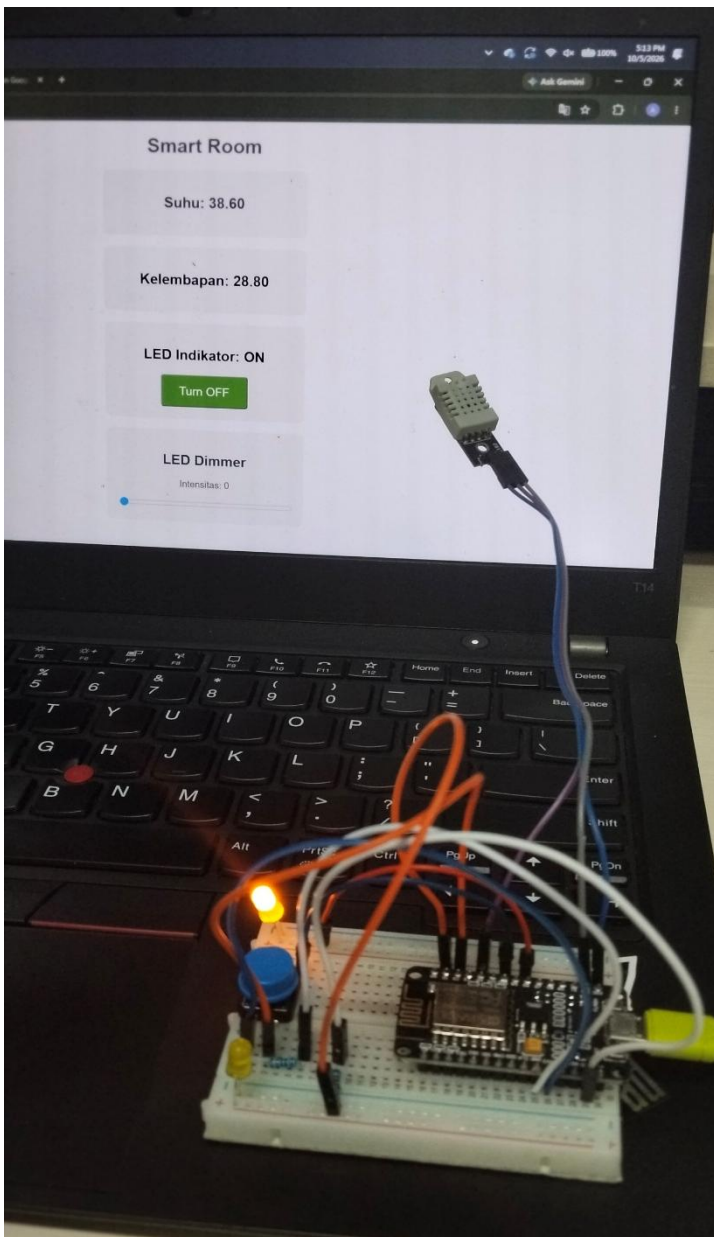
4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via WebSockets

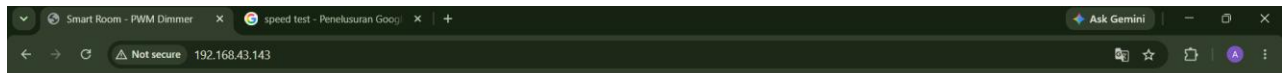
Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (Smart Lighting): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analog `Write()` (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator.
- **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
 - i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
 - ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
 - i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. (*Hint sintaksis:* `<input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">`)

- ii. Buat sebuah fungsi JavaScript bernama `sendPWM(value)` yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
- iii. Pada ruang fungsi C++ `handleWebSocketMessage()` di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
- iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator `analogWrite(pinLED, nilaiPWM)` untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
 - i. Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
 - ii. LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
 - iii. Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.





Smart Room

Suhu: 41.10

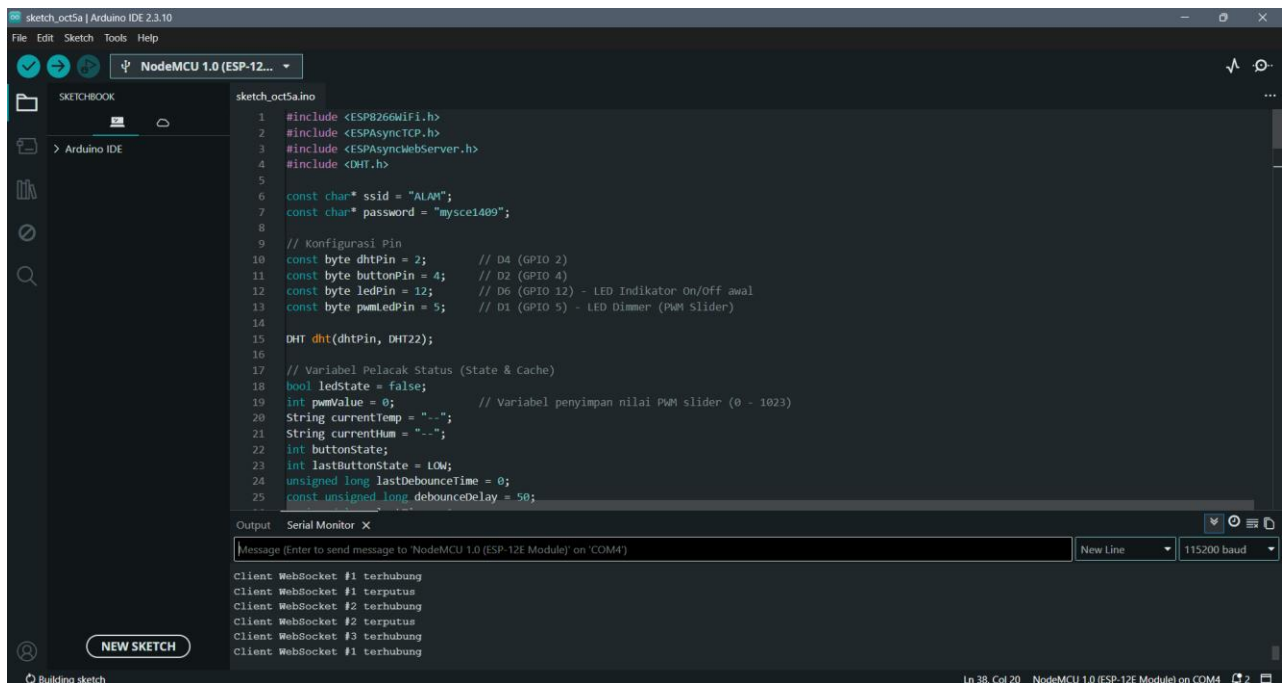
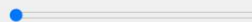
Kelembapan: 25.90

LED Indikator: ON

Turn OFF

LED Dimmer

Intensitas: 0



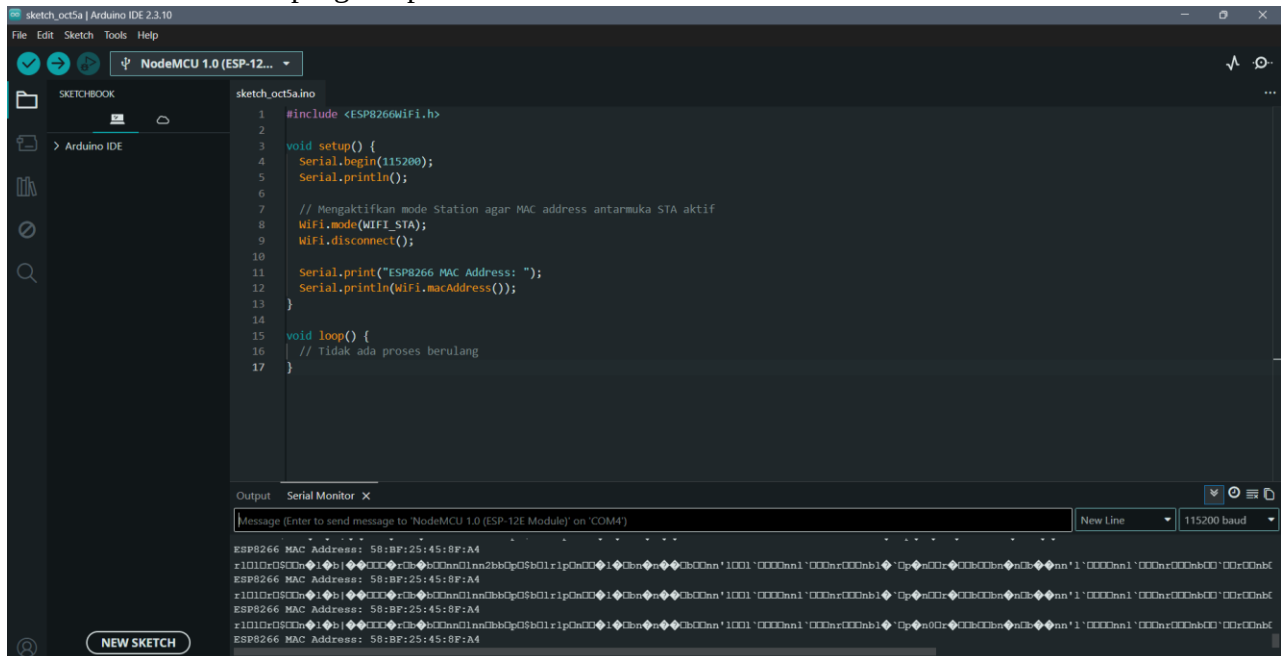
5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (MAC Address) dari masing-masing *board* NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address berikut:



```
1 #include <ESP8266WiFi.h>
2
3
4 void setup() {
5   Serial.begin(115200);
6   Serial.println();
7
8   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
9   WiFi.mode(WIFI_STA);
10  WiFi.disconnect();
11
12  Serial.print("ESP8266 MAC Address: ");
13  Serial.println(WiFi.macAddress());
14
15 }
16
17 void loop() {
18   // Tidak ada proses berulang
19 }
```

3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90).
6. Beri label fisik pada *board* (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

Penjelasan Singkat Kode:

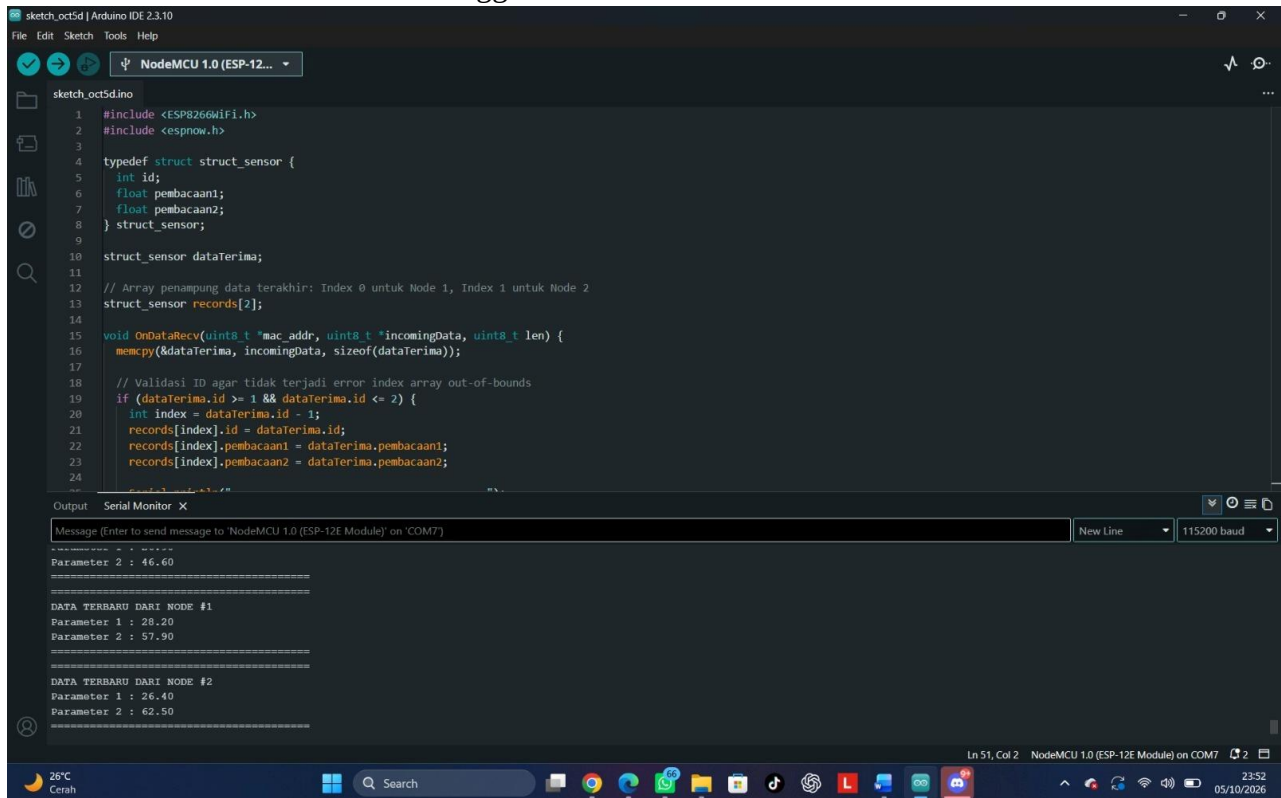
- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).
3. Tuliskan kode berikut dan unggah ke **NodeMCU 1:**



The screenshot shows the Arduino IDE interface with the following code in the sketch editor:

```
1 #include <ESP8266Wifi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (dataTerima.id >= 1 && dataTerima.id <= 2) {
20         int index = dataTerima.id - 1;
21         records[index].id = dataTerima.id;
22         records[index].pembacaan1 = dataTerima.pembacaan1;
23         records[index].pembacaan2 = dataTerima.pembacaan2;
24     }
25 }
```

The serial monitor shows the following output:

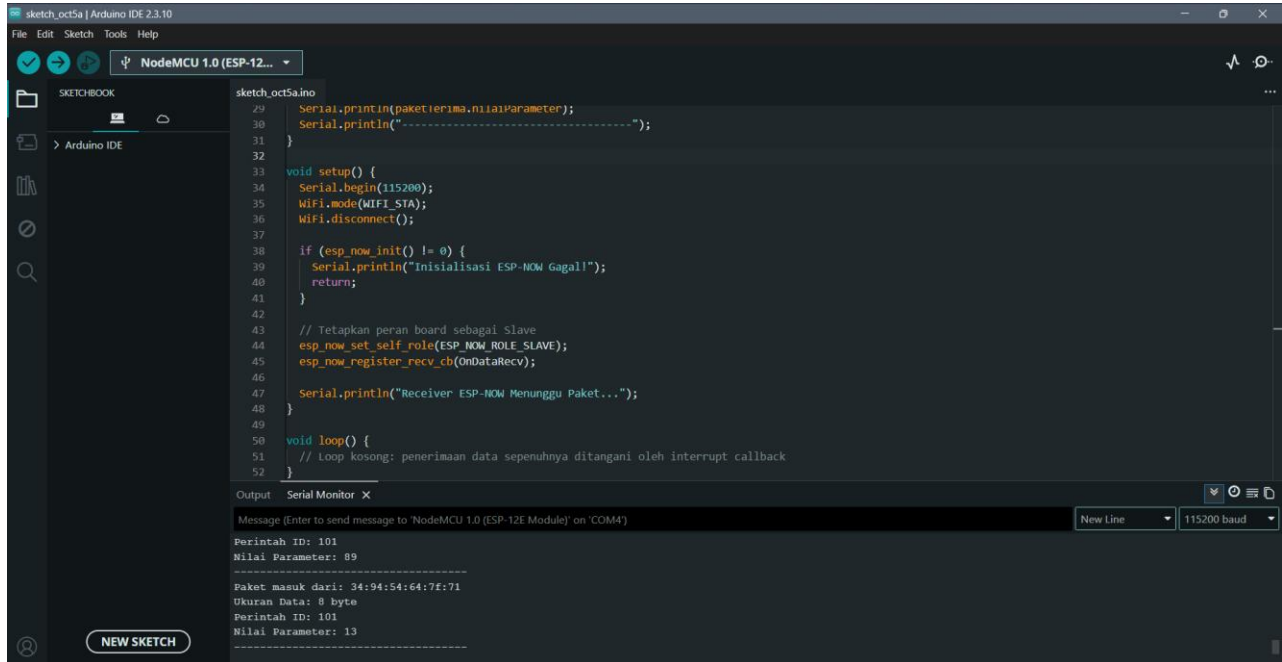
```
-----
Parameter 2 : 46.60
=====
DATA TERBARU DARI NODE #1
Parameter 1 : 28.20
Parameter 2 : 57.90
=====
DATA TERBARU DARI NODE #2
Parameter 1 : 26.40
Parameter 2 : 62.50
=====
```

The status bar at the bottom indicates the board is NodeMCU 1.0 (ESP-12E Module) on COM7, with the cursor at line 51, column 2.

Langkah Kerja Pemrograman Sisi Penerima (Slave):

Unggah kode sketsa berikut ke NodeMCU 2 dan NodeMCU 3.

Board penerima tidak perlu mengetahui alamat MAC milik Controller.

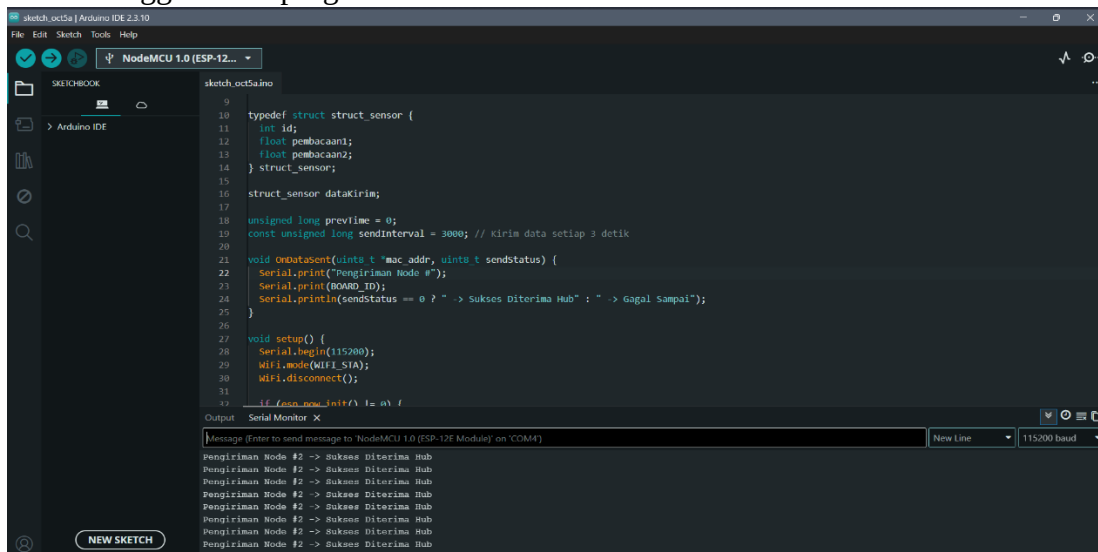


5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

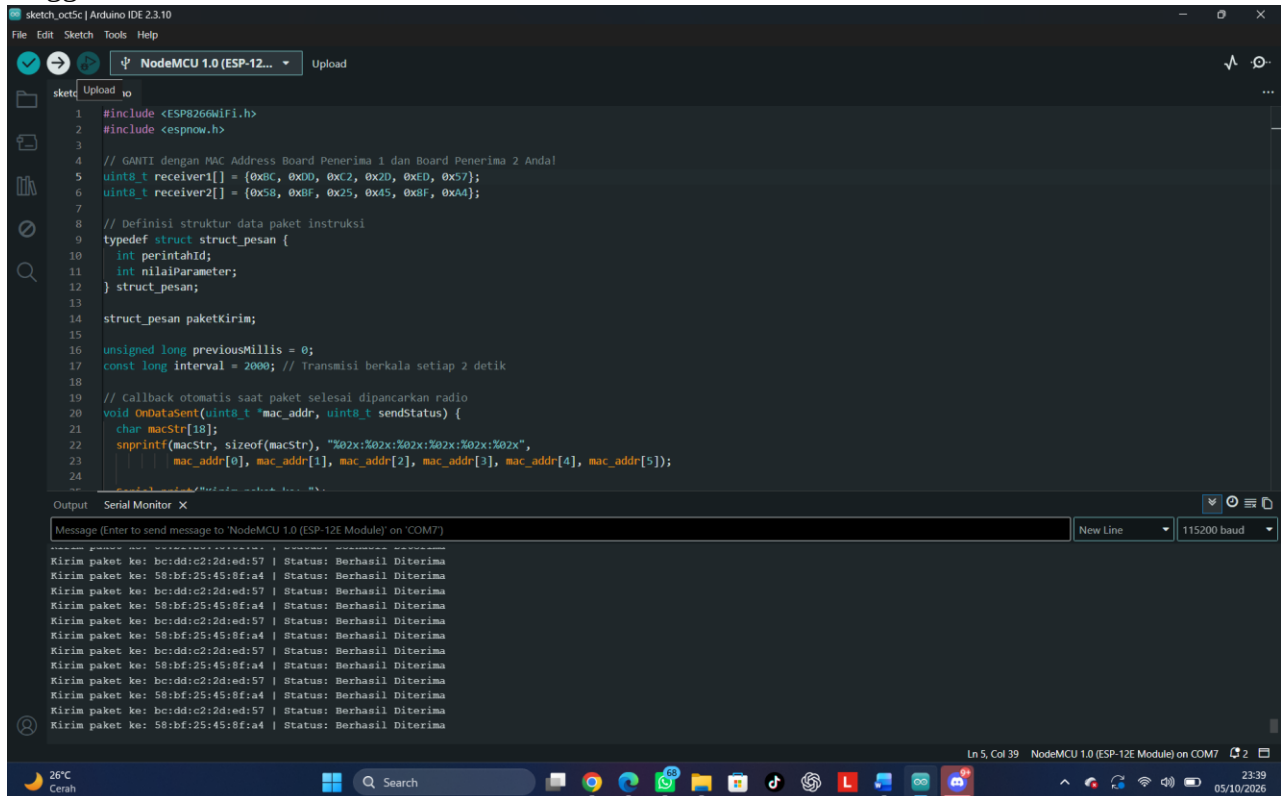
Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik **Board Penerima Pusat (Hub)** pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.
4. Unggah kode program berikut:



Langkah Kerja Pemrograman Sisi Hub Konsentrator: Unggah kode sketsa berikut ke Board Penerima Pusat:



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0x8C, 0xC0, 0xC2, 0xD0, 0xD0, 0x57};
6 uint8_t receiver2[] = {0x58, 0xBF, 0x25, 0x45, 0x8F, 0xA4};
7
8 // Definisi struktur data paket instruksi
9 typedef struct struct_pesan {
10     int perintahId;
11     int nilaiParameter;
12 } struct_pesan;
13
14 struct_pesan paketKirim;
15
16 unsigned long previousMillis = 0;
17 const long interval = 2000; // Transmisi berkala setiap 2 detik
18
19 // Callback otomatis saat paket selesai dipancarkan radio
20 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
21     char macStr[18];
22     sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
23         mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
24
25     Serial.println(macStr);
26 }
27
28 void setup() {
29     Serial.begin(115200);
30     WiFi.begin();
31     while (WiFi.status() != WL_CONNECTED) {
32         delay(500);
33         Serial.print(".");
34     }
35     Serial.println("WiFi connected");
36 }
37
38 void loop() {
39     if (previousMillis + interval < millis()) {
40         paketKirim.perintahId = 1;
41         paketKirim.nilaiParameter = 1;
42         WiFiEspNow.send(receiver1, sizeof(receiver1), &paketKirim, OnDataSent);
43         previousMillis = millis();
44     }
45 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Kirim paket ke: 58:bf:25:45:8f:a4 | Status: Berhasil Diterima

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

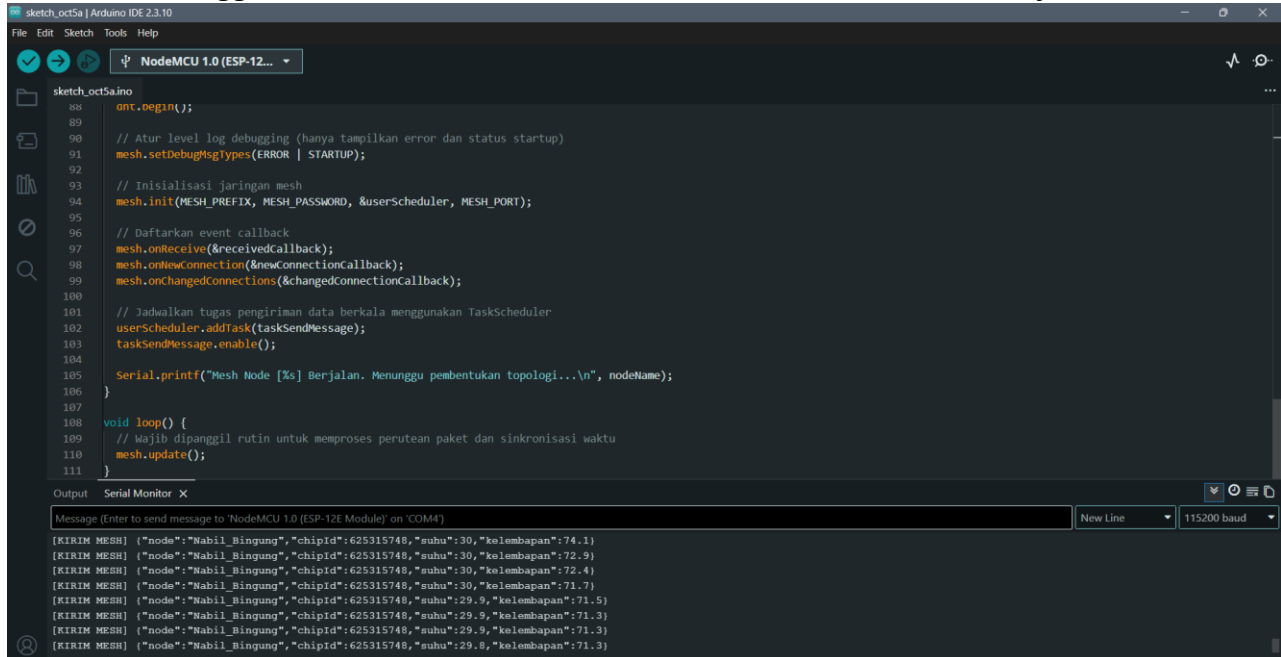
Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor **DHT11** / **DHT22** pada masing-masing board NodeMCU:
 - Pin **VCC** sensor ke pin **3V3 (3.3V)** NodeMCU.
 - Pin **GND** sensor ke pin **GND** NodeMCU.
 - Pin **DATA** / **OUT** sensor ke pin **D4 (GPIO 2)** NodeMCU.

Langkah Kerja Pemrograman:

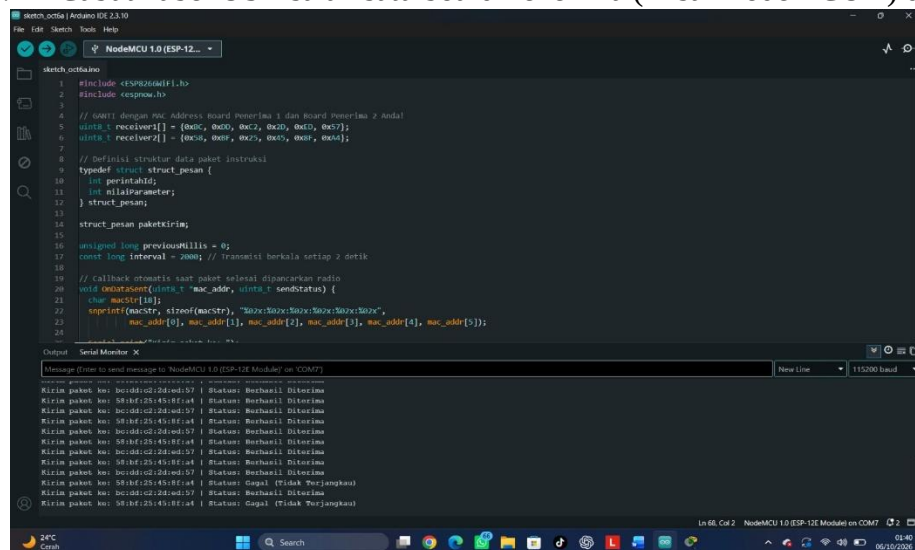
1. Unggah sketsa berikut ke **seluruh board** dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.



.6 Latihan

Kerjakan latihan berbasis eksperimen, observasi serial monitor, dan modifikasi kode berikut untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Sempul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:
 - i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.



- The image shows the Arduino IDE interface with a sketch for a NodeMCU 1.0 (ESP-12E) module. The sketch defines a mesh network with nodes 'Lelaki_Bingung' and 'Nabil_Bingung'. The Serial Monitor shows the output of the sketch, including mesh initialization and node discovery messages.

```
sketch_oct6a.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Lelaki_Bingung" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "Cihuy" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7777 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Gantl DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Nabil_Bingung";
16
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

New Line 115200 baud

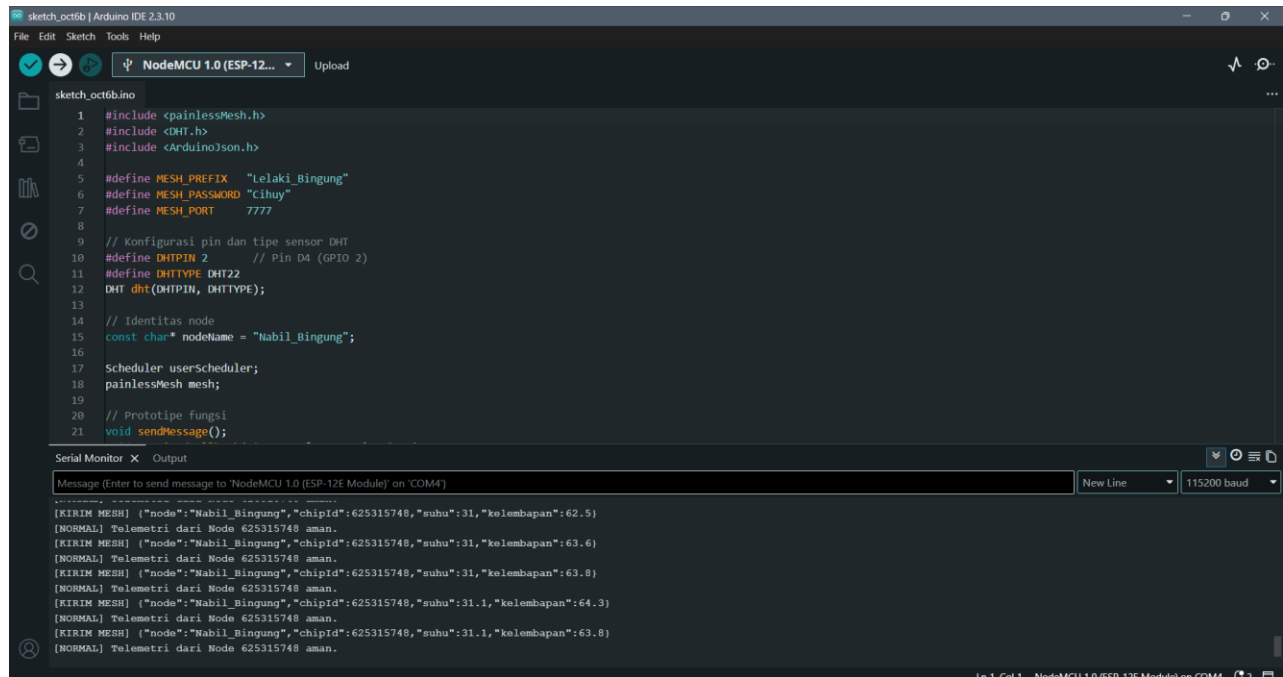
```
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":72.3}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":72.4}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.8}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.6}
r10l0G5000...[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.4}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.4}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.3}
setLogLevel: ERROR | STARTUP |
STARTUP: init(): 1
STARTUP: init(): Mesh channel set to 1
STARTUP: apInit(): AP configured - SSID: Lelaki_Bingung, Channel: 1, IP: 10.237.87.1
STARTUP: apInit(): AP active - Max connections: 4
STARTUP: AP tcp server established on port 7777
Mesh Node [Nabil_Bingung] Berjalan. Menunggu pembentukan topologi...
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.4}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.4}
[KIRIM MESH] {"node":"Nabil_Bingung","chipId":3257789783,"suhu":30.8,"kelembapan":71.3}
```

In 16 Col 1 NodeMCU 1.0 (ESP-12E Module) on COM4

- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!
Jika menggunakan **RST** ataupun menggunakan mencabut kabel daya dia hanya membutuhkan waktu sekitar 0.5 detik untuk membaca Kembali sensor
- **Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.
 - i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas**

32.0 °C (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).

- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Telaki_Bingung"
6 #define MESH_PASSWORD "Cihuy"
7 #define MESH_PORT 7777
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas node
15 const char* nodeName = "Nabil_Bingung";
16
17 Scheduler userscheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi
21 void sendMessage();
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

New Line 115200 baud

```
[KIRIM MESH] ("node":"Nabil_Bingung","chipId":625315748,"suhu":31,"kelembapan":62.5)
[NORMAL] Telemetri dari Node 625315748 aman.
[KIRIM MESH] ("node":"Nabil_Bingung","chipId":625315748,"suhu":31,"kelembapan":63.6)
[NORMAL] Telemetri dari Node 625315748 aman.
[KIRIM MESH] ("node":"Nabil_Bingung","chipId":625315748,"suhu":31,"kelembapan":63.8)
[NORMAL] Telemetri dari Node 625315748 aman.
[KIRIM MESH] ("node":"Nabil_Bingung","chipId":625315748,"suhu":31.1,"kelembapan":64.3)
[NORMAL] Telemetri dari Node 625315748 aman.
[KIRIM MESH] ("node":"Nabil_Bingung","chipId":625315748,"suhu":31.1,"kelembapan":63.8)
[NORMAL] Telemetri dari Node 625315748 aman.
```